PEVG: Planner, Executor, Verifier, Generator

Four Contracts, and the Two Boundaries a Verifier Does Not Both Hold

Nabeel Khan
iSystematic Inc.
ORCID 0009-0005-5364-914X

Pattern Specification, Version 1.0
30 August 2026
doi.org/10.5281/zenodo.22170132
CC BY 4.0

Contents

Abstract **3**

Notation **3**

1. Introduction **4**
1.1 The problem the separation solves . . . 4
1.2 What four contracts buy . . . 4
1.3 Contribution . . . 4
1.4 Scope . . . 4
1.5 Non-goals . . . 4

2. The Separation (Normative) **5**
2.1 Four roles, four contracts . . . 5
2.2 The ordering . . . 5
2.3 What each role may not do . . . 5
2.4 Figure 1: the four contracts . . . 5

3. The Contracts (Normative) **5**
3.1 What a contract states . . . 5
3.2 Outputs are artifacts, not prose . . . 5
3.3 Authority is enumerated, never inferred . . . 6
3.4 Contracts are versioned . . . 6
3.5 Roles may be co-located, contracts may not be merged . . . 6

4. The Four Roles (Normative) **7**
4.1 Planner . . . 7
4.2 Executor . . . 7
4.3 Verifier . . . 7
4.4 Generator . . . 7

5. The Two Boundaries (Normative) **8**
5.1 They are different questions . . . 8
5.2 The collapse error . . . 8
5.3 The requirement . . . 8
5.4 What the separation costs . . . 8
5.5 Figure 2: two boundaries, not one . . . 8

6. What a Verifier Can and Cannot Establish (Normative) **9**

6.1 The verifier is not an oracle . . . 9
6.2 Self-verification is structurally uninformative . . . 9
6.3 Abstention is a distinct outcome . . . 9
6.4 Verification is bounded by what the executor recorded . . . 10

7. Beyond Agent Workflows (Normative Where Marked) 10
7.1 The separation is not specific to agents . . . 10
7.2 What changes when a role is human . . . 10
7.3 The mixed case is the common one . . . 10
7.4 Figure 3: the same contracts, three arrangements . . . 10

8. Evidence (Normative) 10
8.1 Each role records its own . . . 10
8.2 What the executor MUST record . . . 10
8.3 The two boundary decisions are recorded separately . . . 11
8.4 The BOE Declaration . . . 11

9. Limitations and What Has Not Been Tested 11
9.1 The separation has not been evaluated against the alternative . . . 11
9.2 It costs latency and complexity . . . 12
9.3 Four may be the wrong four, and the four are not original . . . 12
9.4 The two-boundary requirement is the least tested part . . . 12
9.5 What would falsify the specification . . . 12
9.6 Research questions . . . 12

10. Versioning 13

11. References 13

Appendix A. Publication Provenance 13

Document History 14

About the Author 14

Licence 14

How to attribute this work 14

Pattern Specification, Version 1.0

Author: Nabeel Khan
ORCID: 0009-0005-5364-914X
Affiliation: iSystematic Inc.
Version: 1.0
Date: 30 August 2026
Status: Public technical specification. Self-declared conformance only. Not a certification scheme.
Licence: Creative Commons Attribution 4.0 International (CC BY 4.0)
DOI (cite this): https://doi.org/10.5281/zenodo.22170132
This version: https://doi.org/10.5281/zenodo.22170133

Cite as: Khan, N. A. (2026). *PEVG: Planner, Executor, Verifier, Generator. Four Contracts, and the Two Boundaries a Verifier Does Not Both Hold. Pattern Specification, Version 1.0.* Zenodo. https://doi.org/10.5281/zenodo.22170132

> **No trademark is claimed on PEVG, on the name of any of the four roles, or on anything else in this document.** The construct is offered for use, teaching, assessment, extension and criticism by anyone, with attribution, under the CC BY 4.0 licence, and no permission beyond that licence is required. PEVG belongs to the engineering family named in Section 11 of the pattern language [1], every name in which is deliberately unmarked. **A mark on a design pattern suppresses the citation the pattern needs in order to spread**, and the defensibility of the name rests on a dated publication record rather than on a symbol.

Relationship to the canonical statement. This specification is the depth treatment of **AP-1** of A Pattern Language for Production LLM Platforms [1]. **That pattern is the canonical statement and governs.** Where this document and AP-1 disagree, AP-1 is correct and this document is defective. The two names denote one construct.

Abstract

An agent that plans a task, calls the tools, checks the result and writes the answer in one undifferentiated step cannot be reasoned about part by part. When it fails, there is no seam to open. The failure is attributed to the agent, which is another way of saying it is not attributed at all, and the remedy applied is usually a change to the prompt, which is another way of saying nobody knows which part was wrong.

PEVG separates that agent into four roles with four declared contracts. A **planner** decomposes the task into ordered steps with explicit dependencies. An **executor** performs tool actions under a capability contract. A **verifier** checks each result before it is believed. A **generator** produces the final response from what survived the check, and asserts nothing the verifier did not pass.

The separation is not modularity for its own sake. It exists so that each role can be measured, bounded and improved on its own, and so that a failure lands somewhere specific: a bad plan, a bad call, a check that passed something it should not have, or an answer that asserted more than the check supported. **Four contracts produce four classes of error rather than one.**

The specification's substantive contribution is the distinction at Section 5, which is the distinction most easily collapsed, because both decisions present as a single question about whether an answer may be used. **The verifier holds the epistemic boundary: what may be believed. It does not thereby hold the operational boundary: what may be disclosed or acted upon.** A claim can be true, correctly verified, and still forbidden to leave the system, because disclosure is governed by classification, authorization and policy rather than by correctness. An implementation must therefore place a policy-enforcement point at the workflow output in addition to the verifier, and record the two decisions separately. **Collapsing them is a single clause performing two functions**, which is exactly the classification error the Boundary Invariant forbids.

Section 7 states the separation's application beyond agent workflows. The four contracts describe any arrangement in which work is planned, performed, checked and reported, including AI-assisted work in which some roles are held by people, and the specification states what changes and what does not when a role is human.

AP-1 carries evidence level E2, which is documented implementation by the author rather than independent production evidence, and Section 9 states what would falsify the separation.

It is a specification, not a certification scheme, and no conformity assessment body operates against it.

Keywords: agent architecture; multi-agent systems; LLM orchestration; verification; separation of concerns; evidence architecture; AI governance; software architecture; design patterns.

Notation

This specification uses **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT** and **MAY** as the normative verbal forms.

- **MUST** indicates a requirement. An implementation that does not meet it is not conformant.
- **SHOULD** indicates a recommendation. Departure requires a recorded rationale.
- **MAY** indicates a permission.

The pattern language [1] uses the verbal forms of ISO/IEC Directives Part 2, in which **shall** carries the meaning assigned to MUST here. The correspondence is exact.

Role. One of the four named responsibilities. A role is not a component: 3.5 states how roles map onto implementations, and one process may hold more than one role provided the contracts remain separately stated and separately recorded.

Contract. The declared inputs, outputs, authority and evidence of a role. **A role without a declared contract is not a PEVG role**, whatever it is called.

doi.org/10.5281/zenodo.22170132

Believed. A claim is believed when the workflow is entitled to build on it or to report it as supported. Belief is what the verifier governs.

Disclosed. A claim is disclosed when it leaves the workflow's trust boundary toward a person or another system. **Disclosure is not governed by the verifier**, per Section 5.

Sections 2 through 8 are normative. Sections 1 and 9 are informative, except where they state a requirement using **MUST**.

---

1. Introduction

1.1 The problem the separation solves

A single agent that does everything is faster to build and is the right choice for many tasks. This specification is not an argument that every agent should be four.

It is an argument about what happens when such an agent is wrong in a way that matters, in a setting where being wrong has consequences. The institution asks which part failed, and the honest answer is that the question has no referent. There was one step. It produced a wrong answer. **The available remedies are to change the prompt, change the model, or add an instruction telling it not to do that again**, and none of the three is diagnosis.

The separation exists to give the question a referent.

1.2 What four contracts buy

Locatable failure. Each role owns a distinct class of error, so a failure lands somewhere: the plan was wrong, the call was wrong, the check let something through, or the answer overstated what the check supported. Four classes of error can be counted separately, which is the precondition for improving any of them.

Independent measurement. A planner can be evaluated on plans against a task set, without running the tools. A verifier can be evaluated on its false-accept and false-reject rates against labelled results. Neither measurement is available when the roles are fused, because there is no intermediate artifact to measure.

Bounded authority. The executor is the only role that touches anything outside the workflow, and it does so under a capability contract. Confining side effects to one role is what makes the workflow's blast radius statable.

1.3 Contribution

1. **Four contracts**, stated with their inputs, outputs, authority, prohibitions and evidence. Section 4.
2. **The two-boundary distinction** at Section 5, with the collapse error named and the requirement for a separate policy-enforcement point.
3. **The verifier's epistemic limits** at Section 6, which is what a verifier cannot establish however well built.
4. **The generalization** at Section 7, to arrangements in which some roles are held by people.

1.4 Scope

Workflows in which a task requires more than one model call, and in which the institution must be able to say afterwards what happened and why. Calibrated to regulated institutions, per the pattern language's scope [1], and usable outside them.

1.5 Non-goals

- **Not a framework or a library.** PEVG is a separation and a set of contracts. It prescribes no orchestration technology.
- **Not a claim that four is the only useful decomposition.** 9.3 states what would show it to be the wrong four.
- **Not a substitute for institutional governance.** Where a workflow's outputs bear on institutional decisions, the governing altitude is MESA [3], and this specification operates below it, as it does below the management-system and risk-management structures at [4] and [5].
- **Not a certification scheme.**
- **Not an empirical claim.** Section 9 states what has not been measured.

---

doi.org/10.5281/zenodo.22170132

2. The Separation (Normative)

2.1 Four roles, four contracts

A conformant workflow MUST separate the four roles, and MUST declare a contract for each, comprising the role's inputs, outputs, authority, prohibitions and evidence.

The requirement is on the contracts rather than on the topology. **Separation is a property of the declarations, not of the process count**, and 3.5 states how roles may be co-located without losing it.

2.2 The ordering

The roles form a cycle rather than a line, because a verifier's rejection returns work to the planner or the executor rather than ending the workflow.

Plan, then execute, then verify, then generate, with rejection returning to the step that can act on it. A conformant workflow MUST define, per rejection class, which role receives the rejection, because a rejection with nowhere to go becomes a retry of the same thing.

2.3 What each role may not do

The prohibitions carry more of the design than the permissions, and each is normative.

- **The planner MUST NOT perform tool actions.** A planner that acts is an executor with an unstated contract.
- **The executor MUST NOT decide whether its own result is correct.** Self-verification is the failure the separation exists to prevent, and 6.2 states why it is not merely weaker but structurally uninformative.
- **The verifier MUST NOT produce the response.** A verifier that writes the answer has an interest in the answer passing.
- **The generator MUST NOT assert anything the verifier did not pass.** This is the contract that makes the chain worth building, and 4.4 states what it means for a claim to be unsupported rather than false.

2.4 Figure 1: the four contracts

---

3. The Contracts (Normative)

3.1 What a contract states

Every role's contract MUST state all five of the following. A contract missing any of them is not a contract, and 2.1 is not satisfied by naming the roles.

#	Element
1	**Inputs**: what the role receives, and from which role
2	**Outputs**: what it emits, in a form the next role can consume without interpretation
3	**Authority**: what it is permitted to touch outside the workflow
4	**Prohibitions**: what it MUST NOT do, per 2.3
5	**Evidence**: what it records, per Section 8

3.2 Outputs are artifacts, not prose

A role's output MUST be an artifact the next role can consume without interpreting it. A plan expressed as a paragraph is a plan the executor must interpret, and an interpretation is an undeclared decision taken by the wrong role.

This is the requirement easiest to relax, because prose passes between roles without any change to the code that moves it, and the relaxation is rarely free. A workflow whose roles exchange prose has four roles and one contract.

doi.org/10.5281/zenodo.22170132

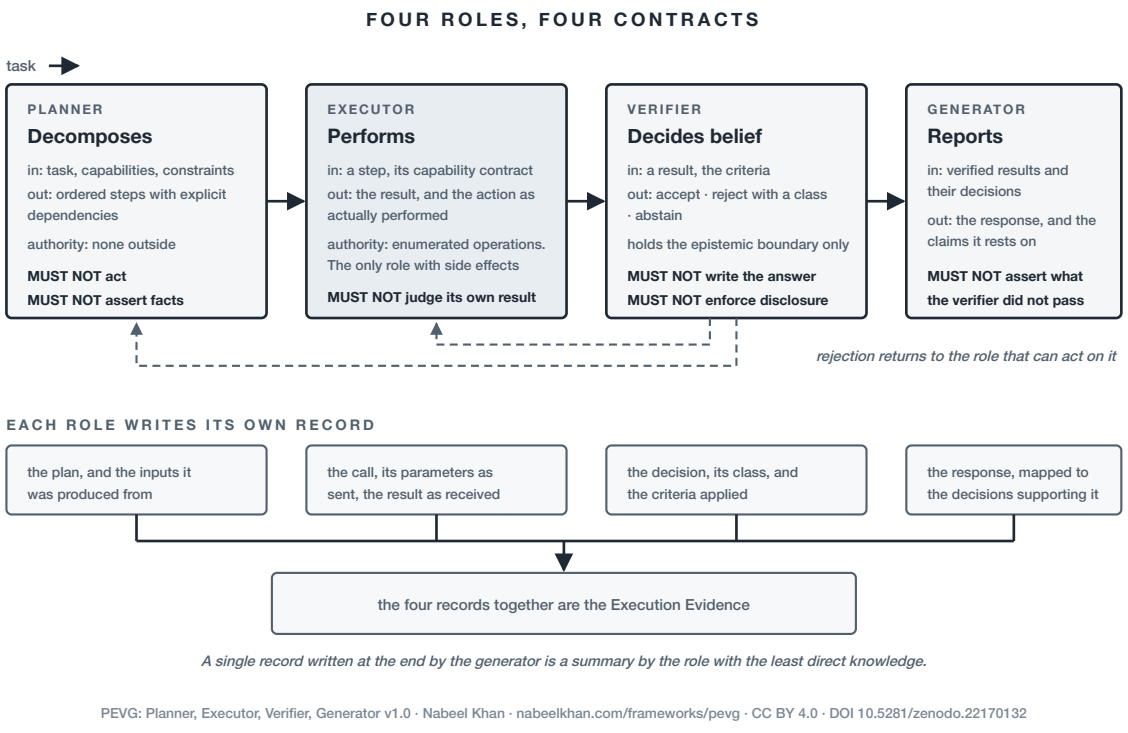


Figure 1. The four contracts. A planner decomposes the task and touches nothing. An executor performs tool actions under a capability contract and is the only role with side effects. A verifier decides what may be believed and produces no output of its own. A generator writes the response from what survived, and asserts nothing more. Rejection returns to the role that can act on it. Each role writes its own evidence, and the four together are the workflow's Execution Evidence.

3.3 Authority is enumerated, never inferred

The executor's authority MUST be an enumerated set of permitted operations, and MUST NOT be expressed as a general permission with exceptions. A capability contract that says what the executor may not do leaves everything else permitted, which is the wrong default at the only role with side effects.

3.4 Contracts are versioned

Each contract MUST carry a version, and the workflow's evidence MUST record which contract versions were in force. A workflow whose behaviour changed because a contract changed, and which cannot say which contract version produced a given result, has evidence that cannot be reconstructed against.

3.5 Roles may be co-located, contracts may not be merged

Two roles MAY be implemented in one process, one model call or one service, provided all of the following hold.

1. The contracts remain **separately stated**.
2. The outputs of each remain **separately recorded**, per Section 8.
3. The prohibitions at 2.3 remain enforced, in particular that **the role deciding correctness is not the role that produced the result**, per 6.2.

The third condition is the one that makes some co-locations impossible. **An executor and a verifier MUST NOT be co-located in a way that lets one model call both produce and accept a result**, because the separation's whole content is that the checking is done by something with no stake in the result passing.

doi.org/10.5281/zenodo.22170132

4. The Four Roles (Normative)

4.1 Planner

Purpose. To convert a task into an ordered set of steps with explicit dependencies.

Inputs. The task, the available capabilities, and the constraints the workflow operates under.

Outputs. A plan: an ordered set of steps, each naming the capability it requires and the steps it depends on. **The dependency structure MUST be explicit** rather than implied by ordering, because an implied dependency cannot be checked and cannot be parallelized safely.

Authority. None outside the workflow.

Prohibitions. MUST NOT perform tool actions. MUST NOT assert facts about the world; a plan states what will be done, not what is true.

Evidence. The plan as emitted, with the contract version and the inputs it was produced from.

4.2 Executor

Purpose. To perform the tool actions the plan requires.

Inputs. A step from the plan, and the capability contract governing it.

Outputs. The result of the action, with the action as actually performed, which MAY differ from the action as planned and MUST be recorded when it does.

Authority. The enumerated operations of its capability contract, per 3.3. **This is the only role with authority outside the workflow.**

Prohibitions. MUST NOT decide whether its own result is correct. MUST NOT perform an operation the contract does not enumerate, including one that appears equivalent to a permitted operation.

Evidence. The call, its parameters, its result, and the capability contract version.

4.3 Verifier

Purpose. To decide what may be believed.

Inputs. A result, the step it was produced for, and the criteria for acceptance.

Outputs. A decision: accept, reject with a class, or **abstain**. **Abstention MUST be available and MUST be distinct from rejection**, because a verifier that cannot say it does not know will say something else, and 6.3 states what abstention means downstream.

Authority. None outside the workflow. **The verifier holds the epistemic boundary and does not hold the operational boundary**, per Section 5.

Prohibitions. MUST NOT produce the response. MUST NOT verify a result it produced. MUST NOT be the policy-enforcement point for disclosure, per 5.3.

Evidence. The decision, its class, the criteria applied, and the contract version.

4.4 Generator

Purpose. To produce the response from what survived verification.

Inputs. The verified results, their verification decisions, and the task.

Outputs. The response, and a statement of which claims it rests on.

Authority. None outside the workflow. Its output passes the policy-enforcement point at 5.3 before leaving.

Prohibitions. MUST NOT assert anything the verifier did not pass.

This prohibition needs one clarification, because it is otherwise read too narrowly. It forbids more than restating a rejected claim. **A generator MUST NOT present an unverified claim with the same confidence as a verified one**, and where the verifier abstained the generator MUST either omit the claim or mark it as unsupported. An

doi.org/10.5281/zenodo.22170132

answer that silently mixes verified and unverified content is the failure mode this contract exists to prevent, and it is invisible to a reader of the answer.

Evidence. The response, and the mapping from its claims to the verification decisions supporting them.

---

5. The Two Boundaries (Normative)

This section is the specification's substantive contribution and the requirement implementations most often miss.

5.1 They are different questions

The epistemic boundary governs *what may be believed*. It is a question about correctness and is the verifier's.

The operational boundary governs *what may be disclosed or acted upon*. It is a question about authorization, classification and policy, and it is not the verifier's.

A claim can be true, correctly verified, and still forbidden to leave the system. A workflow may correctly establish a customer's account balance, a colleague's salary, or the existence of an unannounced acquisition, and be forbidden to disclose any of the three to the party who asked. Correctness has no bearing on the question. **Verifying harder does not help**, because the constraint is not epistemic.

5.2 The collapse error

The characteristic error is to attach the workflow's output boundary at the verifier, on the reasoning that the verifier is already the checkpoint and adding a second one is duplication.

It is not duplication. It is a single clause performing two functions, which is precisely the classification error the Boundary Invariant forbids [1]. The two functions are then optimized together, and the optimization has a direction: a verifier tuned to reduce false rejections of true claims will, by the same movement, reduce refusals to disclose. **Nobody decides to relax the disclosure policy.** It relaxes because it is attached to something else that is being tuned.

5.3 The requirement

An implementation MUST place a policy-enforcement point at the workflow's output, in addition to the verifier, and the two decisions MUST be separately recorded.

The policy-enforcement point:

1. **MUST decide on the basis of classification, authorization and policy**, and MUST NOT take correctness as an input to that decision.
2. **MUST be able to refuse a claim the verifier accepted**, and its refusal MUST NOT be recorded as a verification failure.
3. **MUST record its decision, the policy version in force, and the identity of the party the output is destined for**, since the same claim may be disclosable to one recipient and not another.

5.4 What the separation costs

It costs a second decision point on every output, which is latency, and a second thing to maintain. **An institution for which no output is ever policy-restricted may reasonably decide the cost is not worth paying**, and SHOULD record that decision rather than leaving the absence to be discovered. The requirement at 5.3 applies where any output is restricted.

5.5 Figure 2: two boundaries, not one

---

doi.org/10.5281/zenodo.22170132

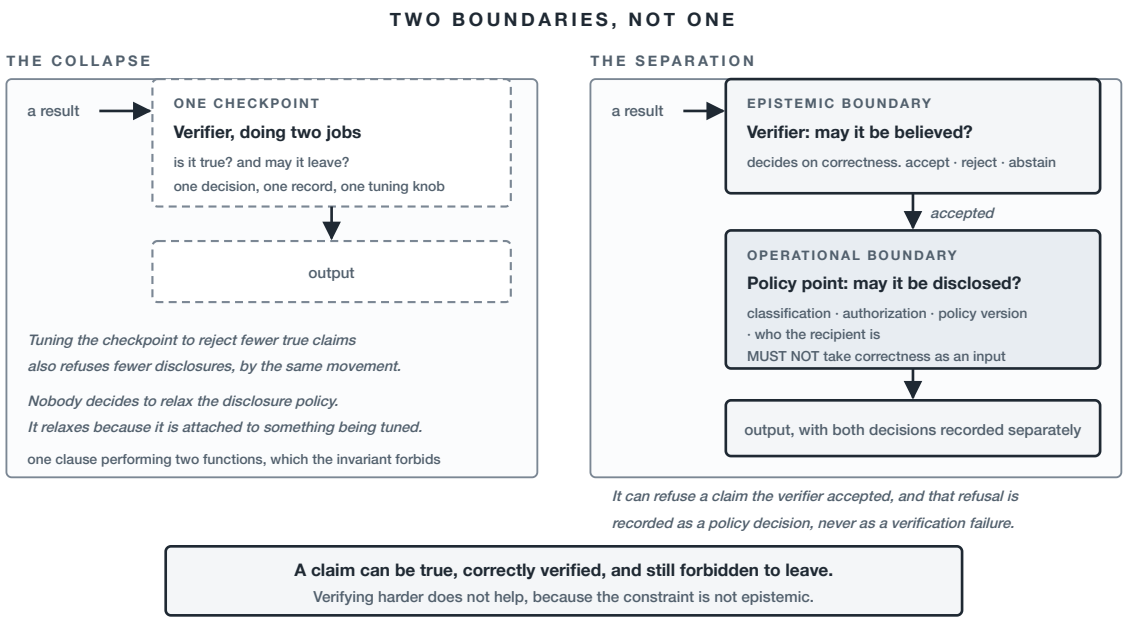


Figure 2. Two boundaries. On the left, the collapse: one checkpoint asked to decide both what is true and what may leave, tuned as one thing, in which relaxing the first quietly relaxes the second. On the right, the separation: the verifier decides what may be believed on the basis of correctness, and a policy-enforcement point decides what may be disclosed on the basis of classification, authorization and the recipient, with its refusal recorded as a policy decision rather than as a verification failure.

6. What a Verifier Can and Cannot Establish (Normative)

6.1 The verifier is not an oracle

A verifier decides whether a result meets stated criteria. **It does not decide whether the result is true**, and a specification that conflated the two would be promising something no implementation can deliver.

A conformant implementation MUST state its verification criteria per step class, so that a reader of the evidence can tell what was actually checked. ”Verified” without stated criteria is a word, not a finding.

6.2 Self-verification is structurally uninformative

A result MUST NOT be verified by the role that produced it, and the reason is stronger than a general preference for independence.

A model asked whether its own output is correct is being asked to sample from the same distribution that produced the output. Where it was wrong for a reason it can detect, it would usually not have been wrong. Where it was wrong for a reason it cannot detect, asking it again does not supply the missing capability. **The check therefore correlates with the thing it is checking**, which is the property that makes an independent check worth having in the first place.

This is why 3.5 permits co-location generally and forbids it for this pair.

6.3 Abstention is a distinct outcome

A verifier MUST be able to abstain, and abstention MUST be carried downstream as its own outcome rather than collapsed into acceptance or rejection.

Downstream, abstention means the generator MUST NOT assert the claim as supported, per 4.4. It MAY report the claim marked as unsupported where the task calls for it. **Collapsing abstention into rejection loses information; collapsing it into acceptance is the failure the contract exists to prevent.**

doi.org/10.5281/zenodo.22170132

6.4 Verification is bounded by what the executor recorded

A verifier can only check what the evidence contains. **Where an executor's record omits the parameters of a call, no verifier can establish that the call was the one the plan required.** The four contracts are therefore mutually dependent in one direction: the verifier's usefulness is bounded by the executor's record, and 8.2 requires the record for that reason rather than for completeness.

---

7. Beyond Agent Workflows (Normative Where Marked)

7.1 The separation is not specific to agents

The four contracts describe any arrangement in which work is planned, performed, checked and reported. Nothing in Sections 3 to 6 requires the roles to be models.

Where a role is held by a person, the contract still applies, and the prohibitions at 2.3 apply unchanged. A human executor MUST NOT verify their own result for the reason at 6.2, which holds for people at least as strongly as for models.

7.2 What changes when a role is human

Three things change, and none of them is the contract.

The evidence is produced differently. A human verifier's decision is recorded rather than emitted, and the recording is a step someone must take. **An implementation MUST NOT treat a human role as exempt from Section 8**, because a human role whose decisions are not recorded is the one gap through which the whole chain becomes unreconstructable.

Abstention becomes more available and less used. A person can say they do not know. Whether they do depends on whether the workflow makes abstention costly, and an implementation SHOULD make it as cheap as acceptance.

The failure classes shift. A human planner produces different bad plans from a model planner. The classes at 1.2 still partition the failures; their frequencies differ.

7.3 The mixed case is the common one

Most real arrangements are mixed: a model plans and executes, a person verifies, a model generates. **The specification is written for that case as much as for the fully automated one**, and an implementation MUST state, per role, whether it is held by a person, a model, or both under stated conditions.

7.4 Figure 3: the same contracts, three arrangements

---

8. Evidence (Normative)

8.1 Each role records its own

Each of the four roles MUST record its own output and its own decisions, and the four records together are the workflow's Execution Evidence in the sense of the pattern language [1].

A single record written at the end by the generator is not Execution Evidence. It is a summary written by the role with the least direct knowledge of what happened, and it cannot support the failure localization at 1.2.

8.2 What the executor MUST record

Named separately because 6.4 depends on it: the operation invoked, its parameters as sent, the result as received, the capability contract version, and the time. **A record of what was requested is not a record of what was performed**, and where the two differ the difference is the most informative thing in the evidence.

doi.org/10.5281/zenodo.22170132

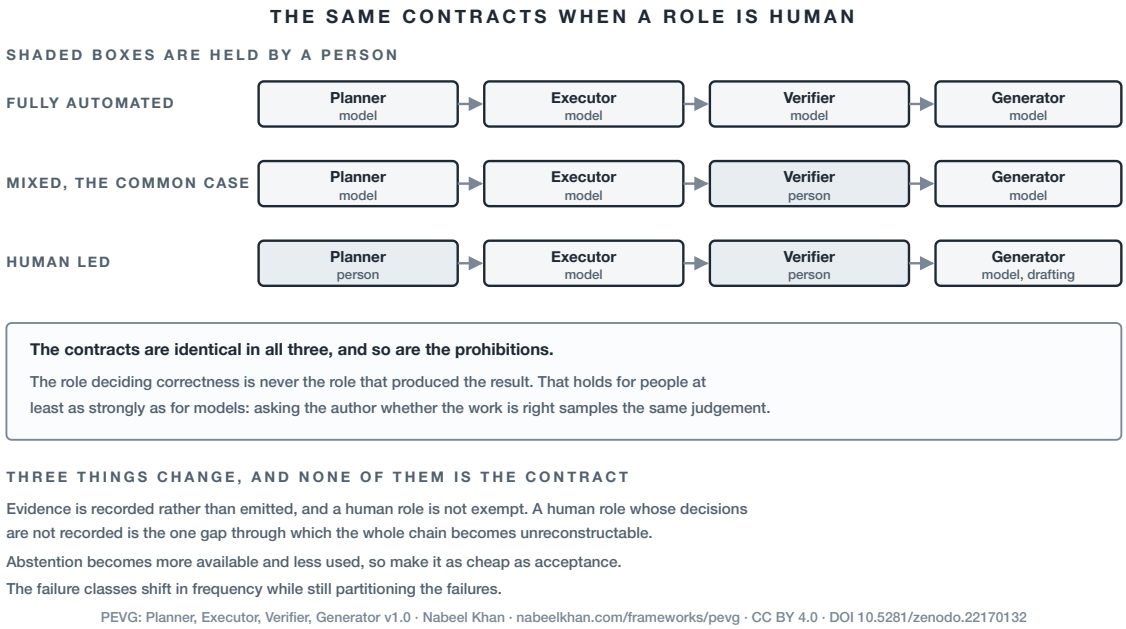


Figure 3. The same four contracts under three arrangements: fully automated, mixed, and human led. The contracts and the prohibitions are identical in all three, including that the role deciding correctness is never the role that produced the result. Three things change when a role is human, and none of them is the contract: the evidence is recorded rather than emitted and the human role is not exempt from recording, abstention becomes more available and less used, and the failure classes shift in frequency while still partitioning the failures.

8.3 The two boundary decisions are recorded separately

Per 5.3. **A single field recording that the output was permitted conflates a verification decision with a policy decision**, and an institution reading that field afterwards cannot tell which of the two it is looking at.

8.4 The BOE Declaration

A PEVG workflow's evidence SHOULD be stated as a BOE Declaration [1]: the boundaries fixed are the verifier's rule on belief and the policy-enforcement rule on disclosure; the optimizer freed is each role, improved on its own; the evidence is the four records and the two boundary decisions. **Stating it in that shape is what lets a workflow's evidence join the institution's other evidence** rather than sitting in its own format.

---

9. Limitations and What Has Not Been Tested

9.1 The separation has not been evaluated against the alternative

The construct stands at evidence level E2: published, implemented in production, and not yet evaluated in an implementation unconnected to the author. AP-1 carries that level in the pattern language [1], which records a production platform operated by the author implementing the separation of Section 5, including the policy-enforcement point at the workflow output. E3, independent production evidence, is the level above it and is not claimed here.

More specifically: **it has not been measured whether a four-role workflow produces fewer or less severe failures than a single-agent workflow of equivalent capability.** The argument in Section 1 is that the separation makes failures locatable, which is a claim about diagnosis rather than about failure rate, and the two are frequently confused in the reader's favour.

doi.org/10.5281/zenodo.22170132

9.2 It costs latency and complexity

Four roles means at least four calls where one might do, and four contracts to maintain. For a task where being wrong is cheap, the separation is overhead. **The specification is calibrated to settings where being wrong is not cheap**, and does not claim to be the right choice elsewhere.

9.3 Four may be the wrong four, and the four are not original

The same four names appear in independent earlier work, and it is stated here rather than left for a reader to find. AgentFlow [6], submitted in October 2025, coordinates four modules it names planner, executor, verifier and generator through an evolving memory, and trains them in the flow of the task. That is earlier than any publication by this author naming the four roles together. **No priority is claimed over it, and the four-role decomposition is not claimed as original.**

The two constructs answer different questions. AgentFlow asks how to train the modules so the system performs better, and optimizes them jointly. This specification asks what each role may not do, and fixes the contracts so that a failure is attributable to one of them afterwards. What is claimed here is narrower than the decomposition: the four declared contracts, and the two-boundary distinction of Section 5, which has no counterpart in that work.

The decomposition is asserted. A reader might reasonably hold that planning and reasoning are distinct, that verification should be split into checking-against-criteria and checking-for-consistency, or that the generator is not a role at all but a formatting step.

None of those objections is answered here. 9.5 states what would settle the question.

9.4 The two-boundary requirement is the least tested part

Section 5 is the specification's substantive contribution and it rests on an argument rather than on evidence. **No study establishes that implementations which collapse the two boundaries actually experience disclosure failures at a higher rate** than those which separate them. The mechanism described at 5.2 is plausible and is not measured.

9.5 What would falsify the specification

PEVG is falsified if four-role workflows are found to produce failures no more locatable than single-agent workflows, measured as the proportion of failures an engineer can attribute to a specific role from the evidence alone. Locatable failure is the separation's central claim, and it is measurable without agreeing on anything else in this document.

Three narrower findings would refute parts of it. If workflows that co-locate the executor and the verifier are found to accept incorrect results no more often than those that separate them, 6.2 is wrong and a large part of the design goes with it. If implementations that attach the output boundary at the verifier experience no more disclosure failures than those with a separate enforcement point, Section 5 is a distinction without a difference. And if a decomposition other than these four localizes failures better, 9.3's objection is sustained.

9.6 Research questions

1. What proportion of failures in a four-role workflow can be attributed to a specific role from the evidence alone, and how does that compare with a single-agent workflow?
2. Do implementations that collapse the two boundaries experience disclosure failures at a measurably higher rate?
3. How often does a verifier abstain when abstention is available, and what happens downstream when it does?
4. In mixed human and model workflows, which role placements produce the best failure localization?

Anyone able to answer any of these from a real implementation is invited to make contact through the ORCID record in the front matter. Question one would move AP-1's evidence level, which is the single most valuable thing that could happen to this construct.

---

doi.org/10.5281/zenodo.22170132

10. Versioning

The major number changes when a change could make a previously conformant workflow non-conformant: a role is added or removed, a contract element is added, or a prohibition at 2.3 is tightened.

The minor number changes for additions and clarifications that cannot.

The four role names are normative and MUST NOT be renamed, because they are the construct's interface to everything that cites it, including the pattern identifier AP-1 [1].

---

11. References

Every reference was verified against a live primary source on 29 August 2026, each with a deliberate failing control, because a check that cannot fail proves nothing.

[1] Khan, N. A. (2026). *A Pattern Language for Production LLM Platforms: Governed Routing, Agent Orchestration, and AI-Native Delivery. Pattern Language Specification.* Zenodo. Concept DOI https://doi.org/10.5281/zenodo.22109864. **AP-1 is the canonical statement of this construct and governs where this document disagrees with it.**

[2] Khan, N. A. (2026). *Prompt Systems and Agent Orchestration: Engineering Multi-Model AI Workflows.* iSystematic Inc. ISBN 978-1-0678317-2-1. Chapter 3 is the source treatment from which this specification is drawn.

[3] Khan, N. A. (2026). *The MESA Framework: A Four-Altitude Diagnostic Model for Institutional AI Governance. Framework Specification.* Zenodo. Concept DOI https://doi.org/10.5281/zenodo.22109836. Cited for the institutional altitude above which this specification does not operate.

[4] International Organization for Standardization and International Electrotechnical Commission (2023). *ISO/IEC 42001:2023, Information technology. Artificial intelligence. Management system.* First edition, December 2023. Cited as the management-system altitude this specification does not duplicate.

[5] National Institute of Standards and Technology (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST AI 100-1, 26 January 2023. Cited for the same reason as [4].

[6] Li, Z., Zhang, H., Han, S., Liu, S., Xie, J., Zhang, Y., Choi, Y., Zou, J., and Lu, P. (2025). *In-the-Flow Agentic System Optimization for Effective Planning and Tool Use.* arXiv:2510.05592. Introduces **AgentFlow**, which coordinates four modules it names planner, executor, verifier and generator through an evolving memory. Submitted 7 October 2025.

Appendix A. Publication Provenance

A.1 First public version of this specification. Not peer reviewed. Deposited on Zenodo, which confers a DOI, a timestamp and citability, and confers no standard status.

A.2 The construct was published before this deposit, as **AP-1** of [1] and at chapter length in [2]. **This document adds depth rather than priority**, and the priority date of the construct is that of its earlier publication rather than of this record. The four-role decomposition is not claimed as original either. AgentFlow coordinates four modules it names planner, executor, verifier and generator, published in October 2025 [6], before any publication by this author naming the four roles together. What this specification claims is narrower: the acronym, the four declared contracts, and the two-boundary distinction of Section 5.

A.3 No preprint identifier exists and no preprint relation is asserted in this record's metadata.

A.4 Publication order. Prepared to publish after version 1.1 of [1], whose Section 11 names this deposit and carries a slot for its DOI. Publishing this record first would leave that slot pointing at nothing for longer than necessary.

A.5 No trademark is claimed on PEVG or on the four role names, which is a decision rather than an omission and is stated in the front matter and at 10.

A.6 The author operates a consulting practice applying this construct, which is a conflict of interest stated here.

Document History

Version	Date	Summary
1.0	30 August 2026	First public version. Four contracts, the two-boundary distinction, the verifier's epistemic limits, the generalization beyond agent workflows, and the evidence requirements

About the Author

Nabeel Khan is an enterprise AI architect and governance advisor, and Founder & Solutions Architect at iSystematic Inc. He has spent twenty-five years architecting enterprise data and machine-learning systems in government, healthcare, financial services and telecommunications across three continents, grounded in TOGAF, DMBOK, ISO 27001 and SOC 2.

He works on governed production AI; systems whose authority is bounded by the architecture, whose decisions carry the evidence needed to defend them, and whose behaviour the institution can explain afterwards. The frameworks specified here, and the regulatory knowledge behind them, are published as machine-accessible AI expertise and are callable over MCP without a key at mcp.nabeelkhan.com/api/mcp, so a reader may query this specification's contents directly and receive answers carrying their source.

ORCID 0009-0005-5364-914X · nabeelkhan.com · Canada

Licence

© 2026 Nabeel Khan. Text licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). **No trademark is claimed on PEVG, on the four role names, or on anything else in this document.**

How to attribute this work

This work is licensed CC BY 4.0. Section 3(a)(1)(A) of that licence requires attribution in the manner the licensor requests, and Section 3(a)(1)(C) requires a link to the material where practicable. This section is that request. Please use one of the following.

Short credit, for slides, diagrams and internal documents:

```
PEVG by Nabeel Khan · https://nabeelkhan.com/frameworks/pevg · CC BY 4.0
```

Inline credit, for web pages, articles and product documentation:

```
Based on <a href="https://nabeelkhan.com/frameworks/pevg">PEVG</a> by
  Nabeel Khan,
licensed under <a href="https://creativecommons.org/licenses/by/4.0/">CC
  BY 4.0</a>.
```

Academic citation:

```
Khan, N. A. (2026). PEVG: Planner, Executor, Verifier, Generator. Four
  Contracts, and the Two Boundaries a Verifier Does Not Both Hold.
(Version 1.0). Zenodo. https://doi.org/10.5281/zenodo.22170132
```

The DOI given is the concept DOI and always resolves to the latest version. Cite it unless you need to pin one specific version.

If you have adapted the work, say that changes were made, and do not imply that this author or iSystematic Inc. endorses your version.

Trademarks. No trademark is claimed on PEVG, on the name of any of the four roles, or on anything else in this document. The name is offered for use without restriction beyond attribution.

doi.org/10.5281/zenodo.22170132